

x86-64 CPU Registerləri

OS (Operating System) Yazmaq üçün Tam Bələdçi

Bu sənəd x86-64 arxitekturasında OS kernel yazan proqramçılar üçün hazırlanmışdır. Hər register üçün həm bit-genişlik cədvəli, həm də OS kontekstində praktiki istifadəsi ətraflı izah edilmişdir. NASM assembly nümunələri real kernel kodundan götürülmüşdür.

1. Ümumi Məqsədli Registerlər (General Purpose Registers)

x86-64 arxitekturasında 16 ümumi məqsədli register var. Onların hər biri tam 64-bit (8 bayt) genişliyindədir, lakin içlərinin 32, 16 və 8 bitlik hissələrinə ayrıca müraciət etmək mümkündür. OS yazanda bu registerlərin ABI (Application Binary Interface) qaydalarına uyğun istifadəsi kritikdir — yanlış istifadə user-space proqramların çöküşünə səbəb olar.

64-bit	32-bit	16-bit	8-bit
--------	--------	--------	-------

RAX	EAX	AX	AL/AH
-----	-----	----	-------

Accumulator — Syscall Nömrəsi + Qayıtma Dəyəri

RAX OS kernel-in ən kritik registeridir. Hər **syscall** çağırışında ikiqat rol oynayır:

Rol 1 — Giriş (syscall-dan əvvəl): Kernel-ə hansı sistem çağırışının istəndiyini bildirir. Məsələn, RAX=1 deməkdir `sys_write`, RAX=60 deməkdir `sys_exit`. Bu rəqəmlər Linux kernel-in syscall cədvəlindədir (`/usr/include/asm/unistd_64.h`). Kernel LSTAR MSR registerindən syscall entry point-ini tapır, oraya atlar və RAX-a baxaraq hansı əməliyyatı edəcəyini müəyyən edir.

Rol 2 — Çıxış (syscall-dan sonra): Kernel nəticəni yenə RAX-a yazır. `sys_write` üçün yazılan bayt sayı, `sys_open` üçün file descriptor, `sys_getpid` üçün prosesin PID-i RAX-a qoyulur. **Xəta halında RAX mənfi olur** (məs. -2 = ENOENT, -13 = EACCES). Buna görə hər syscall-dan sonra xəta yoxlaması şərtdir.

Context switch-də: Kernel hər context switch zamanı bütün registerləri `task_struct`-a saxlayır. RAX da daxildir — proses yarımçıq qalmış syscall-ın qayıtma dəyərini gözləyə bilər.

NASM nümunəsi — `sys_write`:

```

mov rax, 1 ; sys_write syscall nomresi
mov rdi, 1 ; 1-ci argument: stdout (fd=1)
mov rsi, msg ; 2-ci argument: buffer ünvanı
mov rdx, len ; 3-cü argument: bayt sayı
syscall ; kernel-e gec (ring3 -> ring0)
; Kernel basa catdı -- RAX artıq qaytarma deyeridir
test rax, rax ; menfi olarsa xeta var
js .error_handler ; menfidirse xeta!

```

Callee-saved mi? Xeyr — RAX caller-saved-dir. Funksiya çağırılmazdan önce RAX-i stack-e push etmək lazım deyil, amma çağırılan funksiya onu deyisə bilər.

64-bit	32-bit	16-bit	8-bit
RBX	EBX	BX	BL/BH

Base Register — Callee-Saved, Ümumi İstifadə

RBX System V AMD64 ABI-də **callee-saved** registerdir. Bu o deməkdir ki, əgər kernel-in bir funksiyası RBX-i dəyişəcəksə, öncə onu stack-ə saxlamalı, çıxışda bərpa etməlidir. Bu qaydaya əməl edilməsə user-space proqramların RBX-ə güvənən hissələri korlanır.

OS-də istifadəsi: Kernel funksiyalarında uzun müddətli dəyərləri saxlamaq üçün seçilir — məsələn, process descriptor göstəricisi, page table başlanğıc ünvanı. Interrupt handler-lərdə RBX-i mütləq push/pop etmək lazımdır.

```

push rbx ; callee-saved -- saxla
mov rbx, [current_task] ; task pointer-i saxla
; ... kernel isi ...
mov [rbx + offset_pid], eax ; task-in PID-ini yaz
pop rbx ; bərpaa et

```

64-bit	32-bit	16-bit	8-bit
RCX	ECX	CX	CL/CH

Counter Register — Syscall-da RIP Saxlanır

RCX-in OS-də iki kritik rolu var:

Rol 1 — Syscall mexanizmi: CPU *syscall* əmrini icra edəndə avtomatik olaraq **RCX-ə user-space RIP-i (növbəti əmrin ünvanını) yazır**. Kernel qaydanda sysret əmri RCX-dəki ünvana atlar. Buna görə kernel-in syscall handler-i RCX-i pozmamalıdır (əks halda user-space proqram səhv ünvana qayıdar, crash!). Adətən kernel RCX-i dərhal stack-ə saxlayır.

Rol 2 — 4-cü funksiya argumenti: System V ABI-də 4-cü argument RCX-də ötürülür. Lakin syscall-da bu yer R10-a keçir (çünki RCX artıq RIP üçün istifadə olunub).

LOOP əmri: x86-da LOOP əmri RCX-i sayğac kimi istifadə edir — hər addımda RCX azalır, sıfıra çatanda döngü bitir.

```
; syscall entry point-i (kernel terefi)
syscall_entry:
; RCX = user-space RIP (CPU terefinden yazıldı)
; R11 = user-space RFLAGS
push rcx ; user RIP-i yaddasa al
push r11 ; user RFLAGS-i yaddasa al
; ... syscall isi ...
pop r11
pop rcx
sysretq ; RCX-deki unvana qayıt
```

64-bit	32-bit	16-bit	8-bit
--------	--------	--------	-------

RDX	EDX	DX	DL/DH
------------	-----	----	-------

Data Register — 3-cü Arqument, I/O, Bölmə

System V ABI-də **3-cü funksiya argumenti** RDX-də ötürülür. Syscall-da da 3-cü argumentdir (sys_write-da bayt sayı).

I/O portları (OS bootloader-lərdə): in/out əmrləri ilə hardware portlarına müraciət edəndə port ünvanı DX-də olur.

Bölmə əməliyyatı: div/ldiv əmrlərindən sonra qalıq RDX-də qalır (bölen RAX-da, nəticə RDX:RAX cütündə başlayır).

```
; UART (serial port) vasitesile 1 bayt gondermek
mov dx, 0x3F8 ; COM1 port unvani
mov al, 'A' ; gonderilek bayt
out dx, al ; porta yaz
```

64-bit	32-bit	16-bit	8-bit
--------	--------	--------	-------

RSI / RDI	ESI/EDI	SI/DI	SIL/DIL
------------------	---------	-------	---------

Source / Destination Index — Arqumentlər + Yaddaş Əməliyyatları

RDI = 1-ci arqument, RSI = 2-ci arqument (System V ABI). Syscall-da da eyni — sys_write-da RDI=fd, RSI=buffer ünvanıdır.

String əməliyyatları (OS-də çox vacib!): rep movs, rep stos, rep cmps kimi əmrlər RSI-dən (mənbə) RDI-yə (hədəf) data köçürür. Kernel-in memcpy, memset, memcmp implementasiyaları bunlara əsaslanır. Page table-ları sıfırlamaq (page zeroing) üçün rep stosq əmri işlədilir.

```
; 4096 baytliq page-i sifirlamaq (page zeroing)
mov rdi, page_addr ; hedef unvan
xor eax, eax ; sifir deger
mov rcx, 512 ; 512 x 8 bayt = 4096 bayt
rep stosq ; RDI-ye yazir, RDI artir, RCX azalir
```

64-bit

32-bit

16-bit

8-bit

R8–R15

R8d–R15d

R8w–

R8b–

Əlavə Registerlər — Arqumentlər + Kernel Daxili

64-bit rejimdə əlavə edilmiş bu registerlər System V ABI-də 5-ci (R8) və 6-cı (R9) arqumentlər üçündür. R10 syscall-da 4-cü arqument kimi işlənir (RCX yerinə).

R12-R15 callee-saved-dir — RBX kimi, funksiya girişdə saxlamalı, çıxışda bərpa etməlidir.

Kernel optimizasiyaları: Sıx döngülərdə (interrupt handler, scheduler) bu registerlər sıx istifadə olunan dəyişənlər üçün ayrılır ki, yaddaşa müraciət minimuma ensin.

2. Stack və Frame Pointerləri

64-bit	32-bit	16-bit	8-bit
--------	--------	--------	-------

RSP	ESP	SP	SPL
-----	-----	----	-----

Stack Pointer — Kernel Stack vs User Stack, TSS

RSP həmişə stack-in **ən üst elementinin ünvanını** göstərir. push əmri RSP-ni 8 azaldır, pop 8 artırır (x86-64-də stack aşağıya doğru böyüyür).

Kernel stack vs User stack — OS-in ən kritik ayrımı: Hər prosesin **iki ayrı stack-i** var: user-space stack-i (normal C proqramı işlədəndə istifadə olunan) və kernel stack-i (proses kernel kodunu icra edəndə). syscall gəldikdə CPU avtomatik olaraq RSP-ni **TSS (Task State Segment)**-dəki kernel stack ünvanına keçirir. Bu keçid olmadan user-space proqram kernel stack-ini əvəz edə bilər ki, bu da ciddi təhlükəsizlik açığıdır.

TSS qurmaq (OS başlanğıcında vacibdir): Kernel hər CPU üçün bir TSS strukturu yaradır və RSP0 sahəsinə həmin CPU-nun kernel stack ünvanını yazır. Sistem çağırışı gəldikdə CPU TSS.RSP0-dan stack ünvanını götürür.

Interrupt handler-lərdə: Hardware interrupt gəldikdə CPU avtomatik olaraq RFLAGS, CS, RIP (və lazım gəlsə RSP, SS) stack-ə push edir. Kernel interrupt handler-i bu dəyərləri irət əmri ilə bərpa edir.

```
; TSS-de kernel stack-ini qurmaq
mov [tss + 4], rsp ; TSS.RSP0 = kernel stack

; Kernel entry point-inde (syscall geldi)
swapgs ; GS baze adresini user <-> kernel keçir
mov rsp, [gs:kernel_stack] ; kernel stack-e keç
push rcx ; user RIP
push r11 ; user RFLAGS
```

64-bit	32-bit	16-bit	8-bit
--------	--------	--------	-------

RBP	EBP	BP	BPL
-----	-----	----	-----

Base Pointer — Stack Frame Bazası, Debug

RBP bir funksiyanın stack frame-inin başlanğıcını göstərir. Ənənəvi istifadədə hər funksiya girəndə push rbp + mov rbp, rsp edilir. Bu sayədə stack debugger-lər tərəfindən izləyə bilər — RBP zənciri boyunca getməklə call stack-i yenidən qurmaq mümkündür.

OS kernel-də əhəmiyyəti: Kernel panic zamanı stack trace göstərilməsi RBP-yə əsaslanır. GDB, KGDB və digər kernel debugger-lər RBP zəncirindən istifadə edərək hansı funksiyaların çağırıldığını müəyyən edir. -fno-frame-pointer ilə kompilyasiya edildikdə RBP başqa işlər üçün azad olur, lakin debug çətinləşir.

```
func:
push rbp ; kohne RBP-i saxla
mov rbp, rsp ; RBP = hazırki RSP (frame bazi)
sub rsp, 32 ; lokal deyiskenler ucun yer ac
; ...
mov rsp, rbp ; stack-i berpaa et
pop rbp
ret
```

3. OS Kernel üçün Xüsusi Registerlər

64-bit	32-bit	16-bit	8-bit
--------	--------	--------	-------

RIP	—	—	—
-----	---	---	---

Instruction Pointer — Context Switch-in Ürəyi

RIP hər an CPU-nun **növbəti icra edəcəyi əmrin ünvanını** saxlayır. Birbaşa dəyişdirilə bilməz (mov rip, X yanlışdır), yalnız jmp, call, ret, syscall, iret kimi əmrlərlə dəyişir.

Context switch-də: Proses preempt edildikdə scheduler hazırkı RIP-i task_struct-a saxlayır. Proses yenidən işə salındıqda həmin ünvandan davam edir. Bu mexanizm olmadan multitasking mümkün olmazdı.

RIP-relative addressing: x86-64-də Position Independent Code (PIC) üçün [rip + offset] ünvanlaması istifadə olunur. KASLR (Kernel Address Space Layout Randomization) bu xüsusiyyətə əsaslanır.

64-bit	32-bit	16-bit	8-bit
--------	--------	--------	-------

RFLAGS	EFLAGS	FLAGS	—
--------	--------	-------	---

Flags Register — Interrupt Kontrolu, Şərtlər, IOPL

RFLAGS hər arifmetik əməliyyatdan sonra avtomatik yenilənən bit sahələrindən ibarətdir. OS üçün ən vacib bitlər:

IF (Interrupt Flag, bit 9) — ən kritik bit: Bu bit 1 olduqda hardware interrupt-ları CPU tərəfindən qəbul edilir. 0 olduqda masklanır (gözləntiləyə atılır). Kernel-in kritik bölgələrini (spinlock, interrupt handler özü) qorumaq üçün cli (IF=0) və sti (IF=1) istifadə edilir. cli/sti yalnız ring-0 (kernel) rejimdə icazəlidir.

TF (Trap Flag, bit 8): Hər əmrdən sonra debug interrupt (#DB) yaradır. GDB/KGDB single-step rejimi bu bitə əsaslanır.

IOPL (I/O Privilege Level, bit 12-13): User-space proqramların I/O portlarına birbaşa müraciətini idarə edir. OS bunu adətən 0 saxlayır (yalnız kernel I/O portlarına müraciət edə bilər).

CF, ZF, SF, OF — müqayisə (cmp) və şərti atlamalar (je, jne, jg...) üçün istifadə olunur.

```

; Interrupt-lari bağla -- kritik bolge
cli ; IF = 0 (interrupts disabled)

mov [shared_data], rax ; atomik deyil amma interrupt gelmeyecek

sti ; IF = 1 (interrupts enabled)

; Daha yaxsi yol -- RFLAGS saxla/berpaa et
pushfq ; RFLAGS-i stack-e push et
cli
mov [shared_data], rax

popfq ; RFLAGS-i berpaa et (IF kecmish haline donor)

```

64-bit	32-bit	16-bit	8-bit
--------	--------	--------	-------

CR0	—	—	—
-----	---	---	---

Control Register 0 — Paging, Protected Mode, Write Protect

CR0 CPU-nun əsas rejim kontrolu registeridir. OS başlanğıcında bu registerdəki bitlər xüsusi ardıcılıqla qurulmalıdır:

PE (bit 0) — Protected Mode Enable: Bu bit 1 edildikdə CPU real moddan protected modea keçir. GDT (Global Descriptor Table) artıq aktiv olur, segment-based protection işə düşür. Bootloader-in ən ilk işlərindən biridir.

PG (bit 31) — Paging Enable: Bu bit 1 edildikdə CPU bütün yaddaş müraciətlərini page table vasitəsilə çevirir (virtual → fiziki ünvan). PG-ni aktiv etməzdən əvvəl mütləq CR3-ə düzgün page table ünvanı yazılmalıdır — əks halda CPU anlamsız ünvanlara atlar, sistem donur.

WP (bit 16) — Write Protect: Kernel-in read-only olaraq işarələnmiş öz page-lərini dəyişməsinin qarşısını alır. Bu olmadan kernel təsadüfən öz kod bölməsini korlaşdırı bilər. SMEP/SMAP ilə birlikdə kernel security-nin əsasını təşkil edir.

```

; Protected mode-u aktiv et (bootloader-de)
mov eax, cr0

or eax, 0x1 ; PE bitini qur

mov cr0, eax

jmp 0x08:pm_entry ; far jump -- prefetch temizle

; Paging-i aktiv et (kernel başlanğıcında)
mov cr3, rbx ; PML4 table unvani CR3-e yaz

mov rax, cr0

or rax, 0x80000000 ; PG bitini qur

mov cr0, rax ; artıq virtual yaddaş aktıvdir

```


64-bit

32-bit

16-bit

8-bit

CR3

—

—

—

Control Register 3 — Page Table Ünvanı, Proses İzolyasiyası

CR3 hazırkı prosesin **PML4 (Page Map Level 4) cədvəlinin fiziki ünvanını** saxlayır. Bu, x86-64-ün 4 səviyyəli page table iyerarxiyasının başlanğıc nöqtəsidir.

Proses izolyasiyasının əsası: Hər prosesin **öz ayrı CR3**-ü var. Context switch zamanı scheduler yeni prosesin PML4 ünvanını CR3-ə yazır. Bu anında bütün virtual ünvan xəritəsi dəyişir — proses A-nın 0x400000 ünvanı tamamilə fərqli fiziki sahəyə işarə edir, proses B-nin 0x400000-dən isə heç görünmür. Beləliklə proseslər bir-birinin yaddaşına müdaxilə edə bilmir.

TLB (Translation Lookaside Buffer) təmizlənməsi: CR3-ə yazıldıqda CPU avtomatik olaraq TLB-ni (page table cache-ini) təmizləyir. Bu əməliyyat bahalıdır (performance hit). Buna görə Linux PCID (Process-Context Identifiers) istifadə edir ki, hər switch-də TLB-nin hamısı silinməsin.

```
; Context switch -- yeni proses ucun CR3 yaz
mov rax, [new_task + cr3_offset] ; yeni PML4
mov cr3, rax ; TLB temizlendi, yeni page table aktiv

; INVLPG -- tekce bir unvani TLB-den sil (seli effektiv)
invlpg [virtual_addr]
```

64-bit

32-bit

16-bit

8-bit

CR4

—

—

—

Control Register 4 — Təhlükəsizlik, Genişlənmiş Xüsusiyyətlər

CR4 müxtəlif CPU xüsusiyyətlərinin aktiv/deaktiv edilməsini idarə edir:

SMEP (bit 20) — Supervisor Mode Execution Prevention: Kernel-in user-space yaddaşındakı kodu birbaşa icra etməsinin qarşısını alır. Bu olmadan, haker user-space-ə shellcode yerləşdirib kernel-i oraya atlamağa məcbur edə bilər (classic exploit texnikası).

SMAP (bit 21) — Supervisor Mode Access Prevention: Kernel-in user-space yaddaşını birbaşa oxumasının/yazmasının qarşısını alır. Kernel user data-ya müraciət etmək üçün xüsusi STAC/CLAC əmrləri istifadə etməlidir.

PAE (bit 5) — Physical Address Extension: 32-bit rejimdə 4 GB-dan çox fiziki yaddaşa müraciəti aktiv edir. 64-bit rejimdə artıq vacib deyil.

PSE (bit 4) — Page Size Extension: 4 KB yerinə 2 MB (large page) istifadəsinə imkan verir. Kernel-in özü üçün large page-lər TLB miss-lərini azaldır.

```
; SMEP ve SMAP-i aktiv et (kernel init-de)
```

```
mov rax, cr4
```

```
or rax, (1 << 20) ; SMEP
```

```
or rax, (1 << 21) ; SMAP
```

```
mov cr4, rax
```

4. Segment Registerləri

64-bit	32-bit	16-bit	8-bit
--------	--------	--------	-------

CS / SS	—	—	—
---------	---	---	---

Code/Stack Segment — Ring Səviyyəsi (Privilege Level)

64-bit uzun rejimdə segment-əsaslı ünvanlamanın əksər hissəsi aradan qalxıb (baza=0, limit=maksimum). Lakin CS registeri hələ də kritikdir çünki **CPL (Current Privilege Level)** onun ən aşağı 2 bitindədir.

CPL = 0 (Ring 0): Kernel rejimi — CR0, CR3, MSR kimi qorunan registerlərə müraciət mümkündür. cli/sti icazəlidir. **CPL = 3 (Ring 3):** User rejimi — privileged əməllər #GP fault yaradır.

Syscall/sysret əməlləri CS-i avtomatik dəyişir: STAR MSR-də kernel CS segmenti (ring-0) və user CS segmenti (ring-3) yazılır. syscall CS-i kernel segmentinə keçirir (CPL=0), sysret user segmentinə qaydır (CPL=3).

64-bit	32-bit	16-bit	8-bit
--------	--------	--------	-------

FS / GS	—	—	—
---------	---	---	---

FS / GS — Per-CPU Data və Thread Local Storage (TLS)

64-bit rejimdə FS və GS-in baza ünvanları (FS.base, GS.base) MSR vasitəsilə qurula bilər. Bu onlara xüsusi bir qabiliyyət verir: sıfırdan uzaq böyük ünvanlara (fiziki yaddaşın istənilən yerinə) göstərici kimi istifadə.

GS — Per-CPU kernel data (OS üçün çox vacib): Linux kernel-i hər CPU üçün ayrı bir *percpu* data strukturu saxlayır. GS.base həmin CPU-ya aid strukturun ünvanına qurulur. Bu sayədə kernel hangi CPU-da işlədiyindən asılı olmayaraq [gs:offset] ilə həmin CPU-nun öz datalarına (current task, kernel stack, interrupt counter) müraciət edə bilər.

swapgs əmri: Syscall/interrupt gəldikdə CPU hələ user-space GS dəyərinə malikdir. swapgs MSR_KERNEL_GS_BASE ilə cari GS.base-i dəyişir — beləliklə kernel öz per-CPU strukturuna müraciət edə bilər. Sysret-dən əvvəl yenə swapgs edilir.

FS — Thread Local Storage (TLS): User-space-də hər thread öz FS.base ünvanını pthread vasitəsilə qurur. Bu sayədə hər thread-in öz ayrı errno, thread-local dəyişənləri olur.

```
; Syscall entry-de (kernel teref)
swapgs ; user GS <-> kernel GS (MSR_KERNEL_GS_BASE)
mov rsp, [gs:0] ; kernel stack unvani per-CPU datadan

; Kernel icinde -- hansı CPU-da olduğumu tap
mov eax, [gs:cpu_id_offset]
```

5. MSR (Model Specific Registers) — Kernel Başlanğıcı üçün

MSR-lər rdmsr (oxu) və wrmsr (yaz) əməlləri ilə müraciət edilir. Bu əməllər yalnız ring-0-da (kernel) icazəlidir. ECX-ə MSR nömrəsini yazıb rdmsr/wrmsr çağırılır — nəticə EDX:EAX cütündə gəlir.

MSR Adı	Nömrə	OS-də İstifadəsi
EFER (Extended Feature Enable)	0xC0000080	SCE biti (bit 0): syscall/sysret aktiv et LME biti (bit 8): Long Mode (64-bit) aktiv et NXE biti (bit 11): No-Execute page-lar üçün
STAR (Syscall Target)	0xC0000081	Bit 32-47: kernel CS (syscall-da CS olacaq) Bit 48-63: user CS (sysret-de CS olacaq) Syscall-da CPU STAR-dan CS/SS dəyərlərini götürür
LSTAR (Long Mode Syscall Target)	0xC0000082	64-bit syscall handler-in ünvanı. User-space syscall etdikdə CPU bu ünvana atlar. Kernel-in entry point-i buraya yazılır.
SFMASK (Syscall Flag Mask)	0xC0000084	syscall əmrindən sonra RFLAGS-dan silinəcək bitlər. Adətən IF (bit 9) silinir — kernel-in başlanğıcı interrupt-suz keçir.
FS.base / GS.base	0xC0000100 0xC0000101	FS və GS baza ünvanları (user-space TLS üçün). wrfsbase/wrgsbase əməlləri daha sürətlidir (CR4.FSGSBASE lazımdır).
KERNEL_GS_BASE	0xC0000102	swapgs əmri GS.base ilə bu MSR-i dəyişir. Kernel per-CPU ünvanı buraya yazılır, user-space GS buraya swap edilir.

```
; LSTAR-a kernel syscall handler-i yaz (kernel init-de bir defe)
mov ecx, 0xC0000082 ; LSTAR MSR nomresi
mov rax, syscall_entry ; handler unvani (asagi 32 bit)
mov rdx, syscall_entry ; handler unvani (yuxari 32 bit)
shr rdx, 32
wrmsr ; LSTAR = syscall_entry

; EFER -- SCE (syscall enable) bitini qur
mov ecx, 0xC0000080 ; EFER
rdmsr
or eax, 1 ; SCE bit
wrmsr
```

6. Linux Syscall Konvensiyası (System V AMD64 ABI)

Aşağıdakı cədvəl Linux-da syscall çağırışında hər registerin rolunu göstərir. Funksiya çağırışından fərqlidir — 4-cü argument RCX deyil, R10-dadır (çünki syscall əmri RCX-ə user RIP-i yazır).

Register	Rol	Nümunə (sys_write)
RAX	Syscall nömrəsi → qayıtma döyürü	1 (sys_write) → yazılan bayt sayı
RDI	1-ci argument	fd = 1 (stdout)
RSI	2-ci argument	buf = msg ünvanı
RDX	3-cü argument	count = 13 (bayt sayı)
R10	4-cü argument (RCX yerini!)	—
R8	5-ci argument	—
R9	6-cı argument	—
RCX	CPU syscall-da user RIP-i buraya yazır	Dəyişdirilir — saxlanmamalı
R11	CPU syscall-da user RFLAGS-i buraya yazır	Dəyişdirilir — saxlanmamalı

Callee-saved vs Caller-saved:

Callee-Saved (Funksiya saxlamalıdır)	Caller-Saved (Çağıran saxlamalıdır)
RBX, RBP, R12, R13, R14, R15	RAX, RCX, RDX, RSI, RDI, R8, R9, R10, R11
Funksiya bunları dəyişməklə push/pop etməlidir	Çağıran funksiya bunları itirməyini bilməlidir